

왜 AI 코딩도구를 써야 하는가?

AI를 써도 왜 업무 병목은 달라지지 않는가?

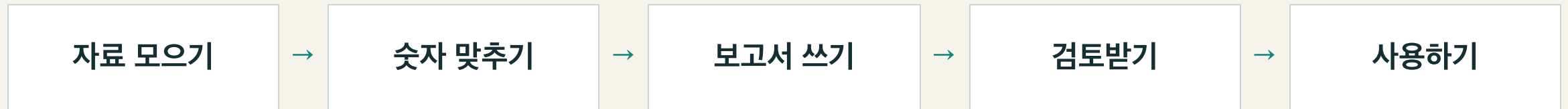
확률과 확실의 세계에서
내 업무를 바꾸는 방법까지

성중 · Delta Society

발표 약 30분 + 실습 45분

좋은 답변을 얻는 것과 업무가 끝나는 것은 다르다

AI가 초안을 만든 뒤에도 남아 있는 일을 보자.



내가 제출한 문제에서, AI의 답변 뒤에도 직접 해야 하는 일은?

우리 업무에는 두 세계가 섞여 있다

같은 납기 관리 업무에도 해석과 규칙 실행이 함께 있다.

확률의 세계

다르게 쓰인 회신을 해석한다.

“다음 주 초”는 언제인가?

어떤 조건을 추가로 확인해야 하는가?

확실의 세계

확인한 값을 정한 규칙으로 처리한다.

날짜순 정렬 · 수량 합산

중복 발주번호 · 필수값 누락 검사

답을 찾아야 하는 부분과, 정한 규칙을 반복해야 하는 부분을 구분한다.

생성형 AI는 맥락에서 가능한 답을 만들어낸다

자연어처럼 표현이 다양하고 모호한 입력을 다루는 데 활용한다.

할 수 있는 일

회신의 뜻 해석

문서에서 필요한 정보 추출

해결 방법과 코드 제안

함께 확인할 것

같은 요청에도 출력이 달라질 수 있다.

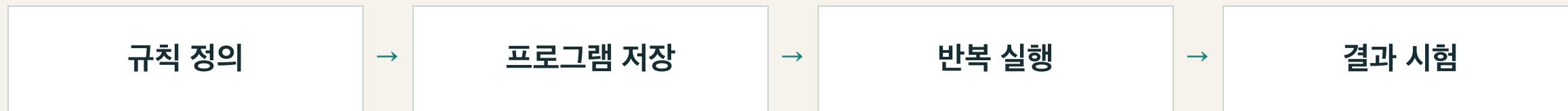
그럴듯함이 정확성을 보장하지 않는다.

근거와 예외를 확인해야 한다.

모호한 내용을 다뤘다면, 불확실한 부분도 결과에 남겨야 한다.

코드는 정한 규칙을 반복 실행할 수 있게 한다

입력과 실행 조건이 같으면 같은 처리를 재현하도록 설계할 수 있다.



매번 부탁하기

“이번에도 빠짐없이 정리해줘.”

절차로 남기기

필수 열 검사 → 중복 검사 → 날짜 정렬 → 파일 저장

여기서 ‘확실’은 무조건 참이라는 뜻이 아니라, 실행을 재현하고 검증할 수 있다는 뜻이다.

반복되는 판단을 확인 가능한 절차로 옮긴다

한 번 얻은 답을 다음 업무에서도 활용할 수 있는 형태로 만든다.

남기는 것

입력 형식과 처리 규칙
실행 가능한 프로그램
기대 결과와 시험 사례

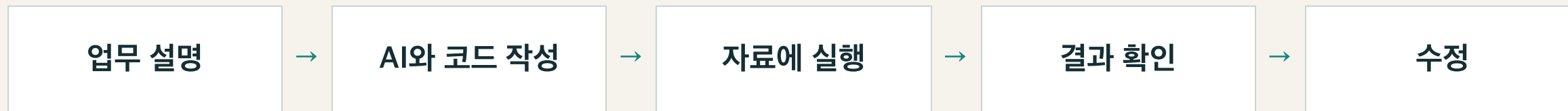
가능해지는 것

새 파일로 같은 절차 재실행
틀린 단계의 원인 추적
규칙 수정 후 다시 시험

반복할 때마다 같은 내용을 설명하고 확인하는 부담을 줄일 수 있다.

업무를 아는 사람이 직접 만들고 시험할 수 있다

AI가 코드 작성을 도와도, 무엇을 만들고 무엇이 맞는지 정해야 한다.



사람이 정할 것

필요한 결과 · 업무 규칙 · 허용할 품질

도구가 도울 것

파일 읽기 · 코드 작성/실행 · 오류 확인/수정

코딩 문법뿐 아니라, 업무를 실행 가능한 규칙으로 설명하는 능력이 중요해진다.

모든 일에 새 프로그램이 필요한 것은 아니다

반복성·규칙·검증 가능성을 보고 수단을 고른다.

코딩도구를 검토할 일

여러 파일을 같은 형식으로 취합
정해진 계산·검사 반복
다음 주에도 같은 절차 재실행

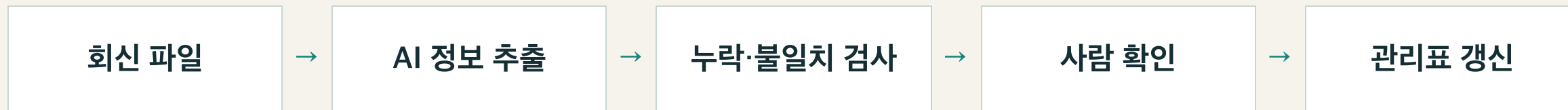
기존 수단으로 충분한 일

일회성 질문과 아이디어 탐색
기존 엑셀 기능으로 해결되는 계산
이미 잘 작동하는 사내 시스템

채팅도 파일·도구 실행을 지원할 수 있다. 핵심은 화면 종류보다 절차를 저장·검증·재사용하는가다.

해석은 AI, 규칙은 코드, 애매한 판단은 확인한다

강의용 일반화 예시 · 공급사 회신을 납기 관리표로 만드는 흐름



AI가 제안

납기일 · 수량 · 조건을 추출하고 원문
근거 연결

코드가 검사

발주번호 중복, 날짜 형식, 필수값 누락
검사

사람이 확인

모호한 날짜, 상충하는 조건, 업무상 예
외

입력 오류·코드 버그는 남을 수 있다. AI 호출이 포함된 단계의 출력 변동성도 사라지지 않는다.

그런데 왜 업무는 여전히 막혀 있을까?

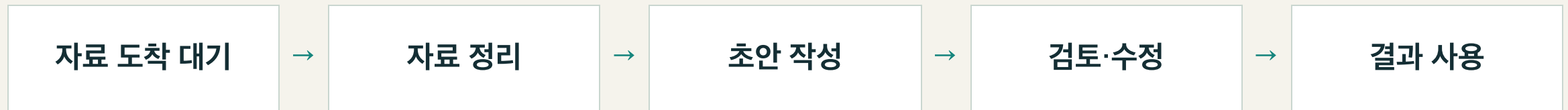
관리표를 빨리 만들었어도 공급사의 회신을 기다리고 있다면?

빨라진 작업이 전체 업무를 제한하는 지점인가?

절약한 개인 시간은 효과다. 다만 그 효과와 전체 업무 완료시간의 변화는 별도로 봐야 한다.

한 단계만 빨라져도 다른 단계는 그대로 남는다

강의용 일반화 예시 · 주간 보고 업무



초안 작성만 개선

자료가 늦고 검토가 밀리면
보고 완료 시각은 크게 달라지지 않을 수 있다.

막히는 원인까지 개선

자료 형식·누락 검사를 앞당기고
검토 기준과 전달 방식을 맞춘다.

어느 쪽이 필요한지는 실제 업무의 대기·재작업·적체를 보고 판단한다.

귀찮은 작업보다 막히는 지점부터 찾는다

참가자가 이미 제출한 문제를 아래 질문으로 다시 읽는다.

어디에 쌓이는가?

처리하지 못한 일이 늘어나는 단계는 어디인가?

무엇을 기다리는가?

자료·회신·확인 중 무엇 때문에 다음 단계로 못 가는가?

왜 다시 하는가?

입력 누락·기준 차이·오류 중 어떤 이유로 되돌아오는가?

처리량을 제한하는 단계와 한 건의 납기를 늦추는 대기는 다를 수 있다. 아직 모르면 병목 후보로 남긴다.

원인이 다르면 만들어야 할 도구도 달라진다

자동화할 기능을 고르기 전에, 무엇 때문에 막히는지 연결한다.

자료 형식이 제각각

공통 형식 변환 + 필수값 검사

동일한 계산을 반복

계산 프로그램 + 결과 대조 시험

검토 때마다 되돌아옴

검토 기준 명시 + 제출 전 검사

자료·권한·협조가 필요

필요한 항목·이유·사용 범위를 정해 요청

자료 접근이 막혀 있다면 획득할 과제로 남긴다. 가능한 입력으로 구현과 시험은 이어간다.

같은 품질에서 무엇이 얼마나 달라졌는가?

서로 다른 사람의 과제 순위보다, 같은 업무의 전후를 비교한다.

사람의 작업시간

준비 + 직접 개입 + 검토 + 수정
AI 대기시간과 구분

업무 완료시간

시작부터 사용 가능한 결과까지
자료·검토 대기 포함

품질과 실제 사용

오류·누락·재작업
실제로 사용한 결과와 완료 건수

도구 사용료와 유지 작업도 함께 기록한다. 자기보고·직접 측정·미확인을 구분한다.

처리할 수 있는 양과 실제로 한 일을 구분한다

시간이 생겼다면 어디에 쓰였는지까지 본다.

가능해진 것

시험에서 더 빨리 처리했다.
추가 업무를 받을 여력이 생겼다.

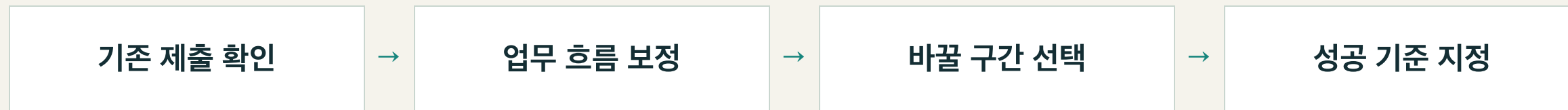
실제로 달라진 것

밀린 업무를 처리했다.
다른 업무로 시간을 옮겼다.
외주·초과근무 지출이 줄었다.

절약한 시간을 임금으로 환산한 값은 실제 현금 절감액과 다르다. 확인한 효과만 구분해서 보고한다.

이미 제출한 문제에서 바꿀 구간 하나를 정한다

오늘의 결과물은 다음 구현 실습에서 사용할 기획서다.



“누가 사용할 결과를, 어디에서 막히는 문제 때문에, 어떻게 바꾸고, 무엇으로 확인하겠다.”

플러그인과 함께 기획서를 구체화한다

플러그인은 근거와 후보를 제안하고, 참가자가 수정·선택한다.

5분 · 불러오기

기존 제출을 읽고 잘못 요약된 내용 수정

10분 · 흐름 정리

사용자·작업·대기·재작업 확인

10분 · 변경 선택

병목 후보를 검토하고 바꿀 작업 지정

10분 · 검증 설계

품질 조건·비교 기준·시험 사례 기록

10분 · 검토와 확정

짝/강사 피드백 반영 후 기획서 저장

내가 만들 도구의 역할을 세 부분으로 나눈다

같은 과제 기록을 이후 회차에서도 이어서 사용한다.

AI가 해석할 부분

모호한 입력은 무엇인가?
근거와 불확실성을 어떻게 남길까?

코드로 고정할 규칙

반복할 계산·검사는 무엇인가?
어떤 사례로 맞는지 시험할까?

사람이 확인할 부분

어떤 예외를 직접 판단할까?
사용 가능한 결과의 기준은 무엇인가?

완료: 사용자가 누구인지, 무엇을 바꿀지, 어떻게 검증할지를 자기 말로 설명할 수 있다.